

GUIDE STANDARD DE PROJET

Symfony 7.4 LTS + Vue 3 + TypeScript

Commandes, démarrage, architecture et conventions d'équipe

PHP	8.3.6
Composer	2.10.2
Symfony CLI	5.17.1
Node.js actuel	18.19.1
npm	9.2.0

Standard recommandé pour tous les nouveaux projets

Version du guide : août 2026

1. Décision technique et compatibilité

Décision principale - Conserver PHP 8.3 et créer les nouveaux backends en Symfony 7.4 LTS. Mettre Node.js à niveau vers la branche LTS 22 avant de créer un nouveau projet Vue/Vite.

- Symfony 7.4 est une version LTS compatible avec PHP 8.2 ou supérieur : PHP 8.3.6 convient.
- Le Symfony courant le plus récent demande PHP 8.4 ou supérieur : ne pas le sélectionner automatiquement avec votre PHP actuel.
- Vite 8 demande Node.js 20.19+ ou 22.12+. Node.js 18.19.1 est donc trop ancien pour un nouveau projet Vue moderne.
- Le standard de ce guide est : Symfony 7.4 LTS, Vue 3, Vite, TypeScript strict, npm et Git.

2. Mise à niveau unique de Node.js

À faire une seule fois sur la machine. NVM permet de changer proprement de version de Node sans modifier les paquets système Ubuntu.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
source ~/.bashrc
nvm install 22
nvm alias default 22
nvm use 22

node -v
npm -v
```

Résultat attendu - Une version Node.js 22.12 ou supérieure. Les projets contiendront ensuite un fichier .nvmrc afin que chaque développeur utilise la même branche Node.

3. Règles d'architecture communes

- Une responsabilité principale par fichier.
- Un nom de fichier explicite et cohérent avec le symbole exporté.
- Pas de dossier générique "utils" rempli de code sans domaine : ranger le code près de la fonctionnalité concernée.
- Contrôleurs minces : validation de la requête, appel du cas d'usage, construction de la réponse.
- Logique métier dans des services/cas d'usage testables, pas dans les contrôleurs ni les composants Vue.
- Les dépendances circulaires sont interdites. Les couches internes ne dépendent pas de l'interface HTTP ou de Vue.
- Les secrets restent dans .env.local et ne sont jamais commités.
- Chaque changement doit passer formatage, analyse statique, tests et build.

À propos de TSX - Dans Vue, le format standard est un composant par fichier .vue. Les fonctions, services, types et composables utilisent des fichiers .ts. TSX est possible, mais il n'est pas retenu ici afin de garder l'écosystème Vue simple et lisible.

4. Choisir le bon type de projet

Besoin	Commande/type	Structure
API, traitement serveur, administration	Symfony seul	backend/

Interface sans backend PHP	Vue seul	frontend/
Application complète séparée	Symfony + Vue	apps/api + apps/web

5. Créer un projet Symfony seul

5.1 Commandes de création

```
mkdir -p ~/Projets
cd ~/Projets

symfony new mon-api --version="7.4.*"
cd mon-api

git init
git add .
git commit -m "chore: initialize Symfony 7.4 project"
```

Pour une application Symfony traditionnelle avec Twig, formulaires et outils web complets :

```
symfony new mon-backoffice --version="7.4.*" --webapp
```

5.2 Dépendances recommandées pour une API

```
composer require symfony/orm-pack symfony/serializer-pack symfony/validator symfony/security-bundle
composer require --dev symfony/maker-bundle symfony/test-pack phpstan/phpstan phpstan/phpstan-symfony friendsofphp/php-cs-fixer
```

5.3 Structure Symfony standard

```
mon-api/  
├── bin/  
├── config/  
├── migrations/  
├── public/  
├── src/  
│   ├── Controller/  
│   ├── Entity/  
│   ├── Repository/  
│   ├── Shared/  
│   └── User/  
│       ├── Application/  
│       │   ├── CreateUser.php  
│       │   └── CreateUserController.php  
│       ├── Domain/  
│       │   ├── User.php  
│       │   └── UserRepositoryInterface.php  
│       └── Infrastructure/  
│           ├── Http/CreateUserController.php  
│           └── Persistence/DoctrineUserRepository.php  
├── tests/  
│   ├── Unit/  
│   ├── Integration/  
│   └── Functional/  
├── .env  
├── .env.local  
├── composer.json  
└── phpunit.xml.dist
```

Principe - Pour un petit projet, Controller/Entity/Repository reste acceptable. Dès qu'un domaine grossit, créer un dossier fonctionnel comme User/, Payment/ ou Contract/ avec Application, Domain et Infrastructure. Ne pas créer vingt couches vides dès le premier jour.

5.4 Démarrer et arrêter Symfony

```
cd ~/Projets/mon-api  
symfony check:requirements  
composer install  
symfony serve -d  
symfony open:local  
  
# Voir les logs  
symfony server:log  
  
# Arrêter  
symfony server:stop
```

5.5 Base de données et migrations

```
# Dans .env.local, définir DATABASE_URL puis :
php bin/console doctrine:database:create
php bin/console make:entity
php bin/console make:migration
php bin/console doctrine:migrations:migrate

# Vérifications
php bin/console doctrine:schema:validate
php bin/console about
```

5.6 Contrôles qualité Symfony

```
composer validate --strict
php bin/console lint:container
php bin/console lint:yml config
php bin/phpunit
vendor/bin/phpstan analyse src tests
vendor/bin/php-cs-fixer fix --dry-run --diff
symfony check:security
```

6. Créer un projet Vue 3 + TypeScript

6.1 Création interactive officielle

```
mkdir -p ~/Projets
cd ~/Projets
npm create vue@latest
```

Réponses standard à utiliser à chaque création :

- Project name : nom-du-projet en kebab-case.
- TypeScript : Yes.
- JSX Support : No.
- Vue Router : Yes pour une application multi-écrans.
- Pinia : Yes dès qu'un état est partagé entre plusieurs écrans.
- Vitest : Yes.
- End-to-End Testing : Playwright pour une application métier importante, sinon No au démarrage.
- ESLint : Yes.
- Prettier : Yes.

```
cd nom-du-projet
printf "22\n" > .nvmrc
nvm use
npm install
npm run dev
```

6.2 Structure Vue standard par fonctionnalité

```
src/
├── app/
│   ├── App.vue
│   ├── router.ts
│   └── main.ts
├── assets/
├── components/
│   └── ui/
│       ├── AppButton.vue
│       └── AppModal.vue
├── features/
│   └── users/
│       ├── api/
│       │   └── fetchUsers.ts
│       ├── components/
│       │   └── UserCard.vue
│       ├── composables/
│       │   └── useUsers.ts
│       ├── pages/
│       │   └── UsersPage.vue
│       ├── stores/
│       │   └── users.store.ts
│       ├── types/
│       │   └── User.ts
│       └── validation/
│           └── userSchema.ts
├── shared/
│   ├── api/httpClient.ts
│   ├── types/ApiError.ts
│   └── validation/
└── tests/
```

Règle de placement - Un élément utilisé uniquement par la fonctionnalité users reste dans features/users. Un composant réellement générique et réutilisé dans plusieurs domaines va dans components/ui ou shared.

6.3 Exemple : une fonction, un fichier TypeScript

```
// src/features/users/api/fetchUsers.ts
import { httpClient } from '@shared/api/httpClient'
import type { User } from '../types/User'

export async function fetchUsers(): Promise<User[]> {
  const response = await httpClient.get<User[]>('/users')
  return response.data
}
```

```
// src/features/users/composables/useUsers.ts
import { onMounted, ref } from 'vue'
import { fetchUsers } from '../api/fetchUsers'
import type { User } from '../types/User'

export function useUsers() {
  const users = ref<User[]>([])
  const isLoading = ref(false)
  const error = ref<string | null>(null)

  async function loadUsers(): Promise<void> {
    isLoading.value = true
    error.value = null

    try {
      users.value = await fetchUsers()
    } catch {
      error.value = 'Impossible de charger les utilisateurs.'
    } finally {
      isLoading.value = false
    }
  }

  onMounted(loadUsers)
  return { users, isLoading, error, loadUsers }
}
```

6.4 Dépendances frontend recommandées

```
npm install axios zod
npm install -D @types/node
```

Installer uniquement selon le besoin :

```
# UI et icônes, facultatif
npm install lucide-vue-next

# Formulaires complexes, facultatif
npm install vee-validate @vee-validate/zod

# Dates, facultatif
npm install date-fns
```

6.5 Variables d'environnement Vue

```
# .env.example
VITE_API_BASE_URL=http://localhost:8000/api

# .env.local (non commité)
VITE_API_BASE_URL=http://localhost:8000/api
```

Sécurité - Toute variable commençant par VITE_ est intégrée au code navigateur. Ne jamais y placer une clé privée, un secret JWT ou un mot de passe.

6.6 Commandes quotidiennes Vue

```
npm install
npm run dev
npm run type-check
npm run lint
npm run test:unit
npm run build
npm run preview
```

7. Créer un projet complet Symfony + Vue

7.1 Structure monorepo recommandée

```
mon-projet/
├── apps/
│   ├── api/          # Symfony 7.4 LTS
│   └── web/          # Vue 3 + TypeScript
├── docs/
│   ├── architecture/
│   └── api/
├── scripts/
├── .editorconfig
├── .gitignore
├── Makefile
└── README.md
```

7.2 Commandes de création

```
mkdir -p ~/Projets/mon-projet/apps
cd ~/Projets/mon-projet

git init

symfony new apps/api --version="7.4.*"

cd apps
npm create vue@latest web
cd web
printf "22\n" > .nvmrc
npm install
cd ../..

git add .
git commit -m "chore: initialize Symfony and Vue applications"
```

7.3 Configuration CORS pour le développement

Dans le backend, installer le bundle CORS si le frontend et l'API tournent sur deux ports différents :

```
cd apps/api
composer require nelmio/cors-bundle
```


Recommandation - Autoriser uniquement l'origine du frontend local en développement, par exemple `http://localhost:5173`. En production, utiliser le domaine exact, jamais une origine universelle avec authentification.

7.4 Lancement dans deux terminaux

```
# Terminal 1 - API
cd ~/Projets/mon-projet/apps/api
symfony serve -d

# Terminal 2 - Frontend
cd ~/Projets/mon-projet/apps/web
nvm use
npm install
npm run dev
```

7.5 Makefile pour uniformiser les commandes

```
# Makefile à la racine
.PHONY: install dev-api dev-web test lint build

install:
    cd apps/api && composer install
    cd apps/web && npm ci

dev-api:
    cd apps/api && symfony server:start

dev-web:
    cd apps/web && npm run dev

test:
    cd apps/api && php bin/phpunit
    cd apps/web && npm run test:unit

lint:
    cd apps/api && vendor/bin/phpstan analyse src tests
    cd apps/web && npm run type-check && npm run lint

build:
    cd apps/web && npm run build
```

Usage - Toute l'équipe utilise les mêmes commandes : `make install`, `make dev-api`, `make dev-web`, `make lint`, `make test` et `make build`.

8. Conventions de nommage

Élément	Convention	Exemple
Dossiers fonctionnels	kebab-case ou nom métier stable	features/payment
Composants Vue	PascalCase	PaymentCard.vue

Composables Vue	use + PascalCase	usePayments.ts
Fonctions TypeScript	camelCase	createPayment.ts
Types/interfaces	PascalCase	Payment.ts
Classes PHP	PascalCase	CreatePaymentHandler.php
Routes API	noms pluriels et versionnés	/api/v1/payments

9. Git et qualité d'équipe

```
# Branches
main
feature/nom-court
fix/nom-court

# Exemples de commits
feat(payment): add payment creation endpoint
fix(auth): reject expired nonce
refactor(user): extract user creation handler
test(contract): cover dispute workflow
chore(deps): update frontend dependencies
```

- Une pull request petite et ciblée vaut mieux qu'une modification massive.
- Le code doit être relu par au moins une autre personne avant fusion.
- Interdire le merge lorsque le build, les tests ou l'analyse statique échouent.
- Committer composer.lock, package-lock.json et symfony.lock.
- Utiliser npm ci en intégration continue et sur les environnements reproductibles.

10. Checklist de création obligatoire

- ☐ Choisir Symfony seul, Vue seul ou monorepo Symfony + Vue.
- ☐ Créer le dépôt Git dès le début.
- ☐ Épingler Symfony 7.4.* avec PHP 8.3.
- ☐ Créer .nvmrc avec 22 pour les projets Vue.
- ☐ Activer TypeScript, ESLint, Prettier et Vitest.
- ☐ Créer .env.example sans secret.
- ☐ Ajouter README.md avec installation et démarrage.
- ☐ Installer uniquement les dépendances réellement nécessaires.
- ☐ Créer les dossiers par fonctionnalité, sans architecture prématurément complexe.
- ☐ Exécuter lint, tests et build avant le premier commit partagé.

11. Commandes de reprise d'un projet existant

11.1 Projet Symfony

```
git clone URL_DU_DEPOT
cd nom-du-projet
composer install
cp .env .env.local
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate
symfony serve -d
php bin/console about
```

11.2 Projet Vue

```
git clone URL_DU_DEPOT
cd nom-du-projet
nvm use
npm ci
cp .env.example .env.local
npm run dev
```

11.3 Monorepo

```
git clone URL_DU_DEPOT
cd nom-du-projet
make install

# Puis dans deux terminaux
make dev-api
make dev-web
```

12. Dépannage rapide

Vite refuse de démarrer

```
node -v
nvm install 22
nvm use 22
rm -rf node_modules
npm ci
npm run dev
```

Symfony sélectionne une version incompatible

```
# Toujours préciser la version avec PHP 8.3
symfony new mon-api --version="7.4.*"
```

Dépendances PHP incohérentes

```
composer diagnose
composer validate --strict
rm -rf vendor
composer install
```

Ports déjà utilisés

```
symfony server:stop
symfony serve --port=8001 -d

# Vue
npm run dev -- --port 5174
```

13. Commandes condensées à copier

Nouveau backend Symfony

```
cd ~/Projets
symfony new mon-api --version="7.4.*"
cd mon-api
composer require symfony/orm-pack symfony/serializer-pack symfony/validator
composer require --dev symfony/maker-bundle symfony/test-pack phpstan/phpstan phpstan/phpstan-
symfony friendsofphp/php-cs-fixer
git init && git add . && git commit -m "chore: initialize project"
symfony serve -d
```

Nouveau frontend Vue + TypeScript

```
cd ~/Projets
npm create vue@latest
cd nom-du-projet
printf "22\n" > .nvmrc
nvm use
npm install
npm install axios zod
npm run dev
```

Nouveau projet complet

```
mkdir -p ~/Projets/mon-projet/apps
cd ~/Projets/mon-projet
git init
symfony new apps/api --version="7.4.*"
cd apps && npm create vue@latest web
cd web && printf "22\n" > .nvmrc && npm install
cd ../../
git add . && git commit -m "chore: initialize full-stack project"
```

14. Sources techniques officielles

Références consultées pour fixer les versions et commandes de ce guide :

- [Symfony - Installing & Setting up the Symfony Framework](#) (documentation officielle, consultée en août 2026).
- [Symfony - Symfony 7.4 Release](#) : version LTS, PHP 8.2 minimum.
- [Vue.js - Quick Start et Using Vue with TypeScript](#) : create-vue et projet Vite prêt pour TypeScript.
- [Vite - Getting Started / Vite 8](#) : Node.js 20.19+ ou 22.12+.
- [TypeScript - option strict](#) : activation des vérifications de types renforcées.